Код до рефакторингу

Food

```
package org.example;

public class Food {
    public String name;
    public int calories;

    public Food(String name, int calories) {
        if (calories < 0) {
            System.out.println("Калорії не можуть бути від'ємними");
            this.calories = 0;
        }
         else {
            this.name = name;
            this.calories = calories;
        }
    }

    public int getCalories() {
        return calories;
    }

    public String getName() {
        return name;
    }

    public void increaseCalories(int extra) {
        calories += extra;
    }
}
```

Бачимо що в коді наявні проблеми з інкапсуляцією, та метод increaseCalories() може призвести до виведення невірних даних (наприклад викликати метод з відємним значенням). Тобто тут доречно використати Encapsulate Field та Remove Dead Code

Exercise

```
package org.example;

public class Exercise {
    public String name;
    public int caloriesBurned;

    public Exercise(String name, int caloriesBurned) {
        if (caloriesBurned < 0) {
            System.out.println("Спалені калорії не можуть бути від'ємними");
            this.caloriesBurned = 0;
        } else {
            this.name = name;
            this.caloriesBurned = caloriesBurned;
        }
    }

    public int getCaloriesBurned() {
        return caloriesBurned;
    }

    public String getName() {
        return name;
    }

    public void decreaseCaloriesBurned(int amount) {
        caloriesBurned -= amount;
    }
}
```

Бачимо що у класі у конструкторі прописано if-else (для перевірки) і можна спростити код використавши метод Math.max (), тобто можна використати Simplify Conditional рефакторинг

Calculate

```
package org.example;
import java.util.ArrayList;
import java.util.List;
public class Calculate {
    public List<Food> foods = new ArrayList<>();
    public List<Exercise> exercises = new ArrayList<>();

    public void addFood(Food food) {
        foods.add(food);
    }
    public void addExercise(Exercise exercise) {
        exercises.add(exercise);
    }
    public int getTotalCalories() {
        int total = 0;
        for (Food food : foods) {
            total += food.calories;
        }
        return total;
    }
    public int getTotalBurnedCalories() {
        int totalBurned = 0;
        for (Exercise exercise : exercises) {
            totalBurned += exercise.caloriesBurned;
        }
        return totalBurned;
    }
    public int calculateNetCalories() {
        int totalCalories = getTotalCalories();
        int totalBurned = getTotalBurnedCalories();
        return totalCalories – totalBurned;
    }
}
```

Бачимо що у класі проблеми з інкапсуляцією, public List<Food> foods та public List<Exercise> exercises можуть бути змінені іззовні, тому доречно використати Encapsulate Collection зробивши поля приватними. Також прямий доступ до полів calories і caloriesBurned, які слід замінити функціями getCalories() та getCaloriesBurned() (Використати Replace Temp with Query). Ну і аналогічно до класів Food та Exercises у класі Calculate доречно використати Math.max() для перевірки від'ємного значення, тобто зробити Simplify Conditional

Код після рефакторингу

Food

```
package org.example;
public class Food {
    private String name;
    private int calories;
    public Food(String name, int calories) {
        this.name = name;
        this.calories = Math.max(calories, 0);
    }
    public String getName() {
        return name;
    }
    public int getCalories() {
        return calories;
    }
}
```

Зробили приватні поля, використали функцію Math.max(), видалили зайву функцію.

Exercise

```
package org.example;

public class Exercise {
    private String name;
    private int caloriesBurned;

    public Exercise(String name, int caloriesBurned) {
        this.name = name;
        this.caloriesBurned = Math.max(caloriesBurned, 0);
    }

    public String getName() {
        return name;
    }

    public int getCaloriesBurned() {
        return caloriesBurned;
   }
}
```

Приватні поля, видалення зайвого коду, використання функції замість if-else операторів.

Calculate

```
package org.example;

import java.util.ArrayList;
import java.util.List;

public class Calculate {
    private List<Food> foods = new ArrayList<>();
    private List<Exercise> exercises = new ArrayList<>();

    public void addFood(Food food) {
        foods.add(food);
    }

    public void addExercise(Exercise exercise) {
        exercises.add(exercise);
    }

    public int getTotalCalories() {
        return foods.stream().mapToInt(Food::getCalories).sum();
    }

    public int getTotalBurnedCalories() {
        return exercises.stream().mapToInt(Exercise::getCaloriesBurned).sum();
    }

    public int calculateNetCalories() {
        int netCalories = getTotalCalories() - getTotalBurnedCalories();
        return Math.max(netCalories, 0);
    }
}
```

Виправлено проблему інкапсуляції, використання Stream API для скорочення коду (замість громістких циклів можно все прописати в 1 рядок, не змінивши функціонал)